



습관 추천 로직 — 기능 명세서 (개발 전달용)

첨부: 습관 추천 로직 DB.xlsx (습관 30개 태깅 데이터), 앱 플로우.mermaid (전체 흐름도) 이 문서는 위 두 파일을 코드로 옮기기 위한 스펙입니다. 머신러닝/추천 알고리즘이 아니라 규칙 기반(rule-based) 필터 체인입니다.

1. 개요

사용자가 (1) 유형 테스트 → (2) 목표 설정 4문항을 마치면, 매일 식단 1개 + 환경 1개 + 운동 1개, 총 3개의 습관을 필터링해서 푸시로 보내주는 기능입니다.

핵심 원칙: - 유형(8종) = “실행 가능한 맥락”을 정하는 필터 - 목표(4종) = “습관의 주제”를 정하는 필터 - 기간(2/4/6주) = “난이도가 올라가는 속도”를 정하는 다이얼 - 식사 시간/외출시간 = “언제 알림을 보낼지”를 정하는 개인화 변수

2. 데이터 모델 (DB 스키마 초안)

2.1 habits (습관 마스터 테이블) — 습관DB 시트 그대로 매핑

컬럼명	타입	설명
id	varchar(4) PK	D01~D10(식단), E01~E10(환경), X01~X10(운동)
category	enum('식단','환경','운동')	
content	text	습관 문구
target_goal	enum('물 자주 마시기','과식하지 않기','술 줄이기','외식/배달음식 줄이기')	
context	enum('무관','회식·외식형','회식·외식·배달형','집·개인공간형','사무실·좌식형')	
difficulty	tinyint (1~3)	
trigger_type	varchar	언제 발동되는 습관인지 (식사시간, 운동, 상시환경 등)
note	text	참고용, 화면 노출 안 함

2.2 cat_types (유형 매핑 테이블) — 유형매핑 시트 그대로 매핑

컬럼명	타입	설명
type_id	tinyint PK	1~8
type_name	varchar	파워 고양이, 인싸 고양이 등
context	varchar	habits.context와 매칭되는 값
priority_goal	varchar	유형 테스트 직후 목표 추천용 (기본 선택값)
start_difficulty	tinyint	해당 유형의 권장 시작 난이도

2.3 users / user_profiles

컬럼명	타입	설명
user_id	PK	
cat_type_id	FK → cat_types	유형 테스트 결과
target_goal	enum	목표 설정 Q1
goal_duration	enum('2주','4주','6주')	목표 설정 Q2
meal_time_breakfast/lunch/dinner	time	목표 설정 Q3
outing_weekday_leave/return	time	목표 설정 Q4
outing_weekend_leave/return	time	목표 설정 Q4
program_start_date	date	난이도 주차 계산 기준일

2.4 daily_habit_logs (완료 기록 — 처음부터 반드시 포함)

컬럼명	타입	설명
log_id	PK	
user_id	FK	
habit_id	FK → habits	
assigned_date	date	오늘 배정된 날짜
category	enum	식단/환경/운동 (완료율 집계용 비정규화)
completed	boolean	사용자 체크 여부
completed_at	datetime nullable	

⚠ 이 테이블을 처음부터 설계에 넣어야 합니다. 나중에 추가하면 난이도 적응 로직(3.4절)을 소급 적용할 데이터가 없습니다.

2.5 difficulty_curve (설정값 테이블, 하드코딩 금지)

컬럼명	타입	설명
duration	enum('2주','4주','6주')	
week_range	varchar	예: "1~2주차"
allowed_difficulty	varchar	예: "1", "1~2", "1~3(조건부)"

난이도커브 시트를 그대로 시드 데이터로 사용.

3. 매칭 엔진 로직 (의사코드)

```
function getTodayHabits(user):
    # 1. context 필터
    userContext = cat_types[user.cat_type_id].context
    pool = habits.filter(h => h.context == userContext OR h.context == "무관")

    # 2. target_goal 매칭
    candidates = pool.filter(h => h.target_goal == user.target_goal)

    # 3. 카테고리별 후보 확인 + fallback
    for category in ["식단", "환경", "운동"]:
        catCandidates = candidates.filter(h => h.category == category)
        if catCandidates.isEmpty():
            # fallback 우선순위 (반드시 이 순서 고정)
            catCandidates = pool.filter(h => h.category == category AND h.context == "무관")
            if catCandidates.isEmpty():
                catCandidates = pool.filter(h => h.category == category) # target_goal 무시, context만 유지

    # 4. 난이도 필터
    week = getCurrentWeek(user.program_start_date)
    allowedDifficulty = difficulty_curve.lookup(user.goal_duration, week)
    catCandidates = catCandidates.filter(h => h.difficulty in allowedDifficulty)

    # 5. 최근 3일 중복 회피
    recentHabitIds = daily_habit_logs.filter(user, last 3 days).map(h => h.habit_id)
    catCandidates = catCandidates.exclude(recentHabitIds)
    if catCandidates.isEmpty():
        catCandidates = (중복 회피 무시하고 재선택) # 후보 고갈 시 예외 처리

    selected[category] = random_or_weighted_pick(catCandidates)
```

```
# 6. 트리거 개인화
for habit in selected.values():
    finalText = personalizeTrigger(habit, user.meal_times, user.outing_times)

return selected # {식단: habit, 환경: habit, 운동: habit}
```

3.1 트리거 개인화 규칙

- 습관 문구에 “식사 30분 전” 같은 상대 시간 표현이 있으면 → `user.meal_time` * 값으로 치환해 알람 시각 계산
- “외출 전 / 회식 전” 표현이 있으면 → `outing_time` 값 사용
- 상대 시간 표현이 없는 습관(“무언가 먹을 때는 식탁 의자에만 앉는다” 등)은 → 하루 중 고정 시간대(예: 오전 9시)에 “오늘의 습관”으로만 안내, 별도 트리거 알람 없음

3.2 난이도 재계산 (주차 전환 시)

```
function recalcDifficulty(user):
    for category in ["식단", "환경", "운동"]:
        completionRate = daily_habit_logs.completionRate(user, category, last 7 days)
        if completionRate >= 0.7:
            categoryDifficultyCap[category] += 1 (최대 3)
            # 낮으면 유지, 낮춘다는 틀은 없음 (하한선 1 고정)
```

3.3 목표기간 종료 시

- `program_start_date` 기준 경과일이 `goal_duration`에 도달하면 리포트 화면 노출
- 리포트에는 카테고리별 완료율, 전체 완료 습관 수 표시
- CTA: “습관 유지하기(동일 설정 반복)” / “새 목표 설정하기(온보딩 재진입)”

4. API 엔드포인트 (제안)

Method	Endpoint	설명
POST	/onboarding/type-test	유형 테스트 결과 제출 → <code>cat_type_id</code> 반환
POST	/onboarding/goal-setting	목표 4문항 제출 → <code>user_profile</code> 생성
GET	/habits/today	매칭 엔진 실행 → 오늘의 습관 3개 반환
POST	/habits/{habit_id}/complete	완료 체크 → <code>daily_habit_logs</code> insert
GET	/habits/report	기간 종료/중간 리포트용 완료율 집계
POST	/user/goal-reset	목표 재설정 (기간 종료 후)

5. 예외/엣지 케이스 (QA 시 꼭 확인)

1. 후보가 fallback까지 다 거쳐도 카테고리당 0개인 경우 → 최소 안전장치로 `context="무관"` 이면서 `target_goal` 무관한 습관 1개를 최종 fallback으로 고정 배정 (habits 테이블에 카테고리별 “무관+무관” 습관이 최소 1개씩은 있어야 함, 현재 30개 데이터 기준 충족 확인됨)
2. `program_start_date` 이후 사용자가 앱을 며칠 안 켜 경우 → 주차 계산은 달력 기준으로 계속 흘러가게 할지, 실제 접속일 기준으로 할지 기획 확정 필요 (미정 항목, 기획 확인 요망)
3. 완료율 계산 시 분모(=배정된 날짜 수)가 0인 신규 유저 → division by zero 방지 처리
4. 유형 재검사를 사용자가 원할 경우 → 기존 진행 중인 `daily_habit_logs`를 유지할지 초기화할지 결정 필요 (미정 항목)

6. 개발 난이도 평가

전체 난이도: 중(Medium) — ML/추천엔진이 아닌 조건문 체인이라 알고리즘 자체는 어렵지 않습니다. 난이도는 대부분 “예외 처리”와 “알림 스케줄링”에 몰려 있습니다.

영역	난이도	이유
DB 스키마 설계	하	테이블 5개 내외, 관계 단순
유형 테스트 / 목표설정 온보딩 UI+API	하	표준 CRUD + 폼
매칭 엔진 (필터 체인)	중	로직 자체는 if문 조합이라 쉽지만, fallback 우선순위와 “후보 0개” 옛 지케이스를 안 챙기면 특정 유형+목표 조합에서 빈 화면이 뜨는 버그가 잘 남
난이도 적응 로직 (완료율 기반)	중	배치 작업(주차 전환 시점에 재계산)이 필요 — cron/스케줄러 설계 필요
트리거 개인화 + 푸시 알림 스케줄링	중~상	사용자별로 다른 시각에 알림을 걸어야 해서, 알림 인프라(FCM/APNs 등) 및 시간대별 발송 큐 설계가 필요. 습관 문구를 파싱해서 “상대 시간 표현”을 실제 발송 시각으로 바꾸는 부분은 처음엔 하드코딩(문구별 트리거 타입을 이미 태깅해뒀으므로 매핑 테이블 방식 추천)으로 단순화 가능
리텐션/리포트 화면	하	집계 쿼리 수준

체감 개발 기간 (1인 기준, 참고용): - 백엔드(스키마+매칭엔진+API): 1~2주 - 알림 스케줄링(개인화 트리거 포함): 1주 - 프론트(온보딩+오늘의 습관 화면+리포트): 1~2주 - QA/옛지케이스 보정: 3~5일

→ 총 4~6주 내외면 MVP 개발이 가능한 범위입니다. 다만 “습관 문구 파싱해서 시간 자동 치환” 부분을 정교하게 하려 하면(자연어 처리) 난이도가 급상승하니, **처음엔 trigger_type을 이미 데이터에 태깅해둔 방식(3.1절)으로 매핑 테이블 기반 단순 치환**을 권장합니다. 이 방식이면 알림 스케줄링도 “상”이 아니라 “중” 수준으로 내려갑니다.

개발 착수 전 기획에서 먼저 확정해야 할 것 (5절 미정 항목들): - 며칠 미접속 시 주차 진행 기준 (달력 vs 접속일) - 유형 재검사 시 기존 로그 처리 방식 - fallback 최종 단계에서도 후보가 없을 때의 디폴트 습관(카테고리별 최소 1개 “만능 습관” 지정 필요)

이 세 가지를 먼저 정해서 넘기면, 개발자가 중간에 다시 물어보는 왕복을 줄일 수 있어요.